Republic of the Philippines
Bicol University Polangui
Polangui Albay

LAB REPORT
(MERGE SORTING)

SUBMITTED BY:

GIAN CARLO TRILLES
BScPE-2A

SUBMIITED TO:

MELISSA JEANKIE RELLON
INSTRUCTOR

DATE:
OCTOBER 10,2024

Machine Problem Description: Faulty Machine Finder in a Factory

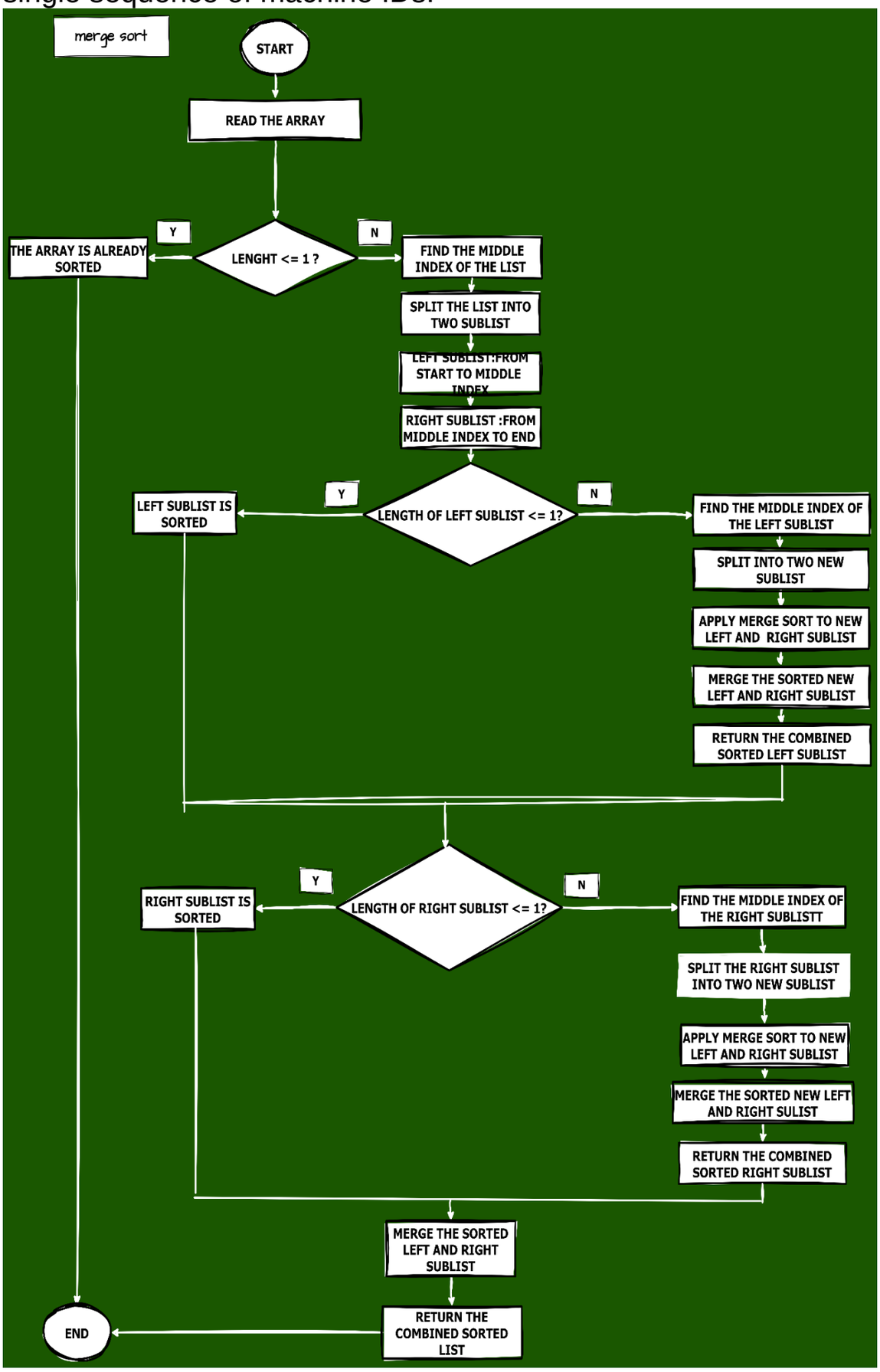
In a factory, multiple machines are producing items. Each machine is identified by a unique ID, and most machines produce an even number of items, meaning their IDs appear twice in the list of machines. However, one machine is faulty and produces an odd number of items, which means that the faulty machine's ID appears only once in the list. The goal is to sort the list of machine IDs using different sorting algorithms and then identify the faulty machine by applying the **XOR operation** to find the ID that appears only once.

MERGE SORT:

This is a divide-and-conquer algorithm that splits the array into two halves and recursively sorts both halves. After this, merge the two sorted halves together.

I FLOWCHART (MERGE SORT):

In the following flowchart, the program uses Merge Sort on a single sequence of machine IDs.



SOURCE CODE:

In the following code, the program uses Quick Sort on a single sequence of machine IDs.

FaultyMachine_MergeSorting.py

```python
def mergeSort(machines):  4 usages
    if len(machines) <= 1:
        return machines
    mid = len(machines) // 2
    left = mergeSort(machines[:mid])
    right = mergeSort(machines[mid:])
    return merge(left, right)
#combine thetwo sorted array to singlse sorted array
def merge(left, right):  1 usage
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def findFaultyMachine(machines):  1 usage
    result = 0
    for machine in machines:
        result ^= machine
    return result
def main():  1 usage
    machines = [303, 101, 404, 202, 101, 303, 404]
    print(f"Original machines: {machines}")
    sorted_machines = mergeSort(machines)
    sorted_machines = mergeSort(machines)
    print(f"Sorted machines using merge sort: {sorted_machines}")
    faulty_machine = findFaultyMachine(machines)
    print(f"Faulty machine: {faulty_machine}")
if __name__ == "__main__":
    main()
```

OUTPUT:

In this program, the machine IDs are sorted in ascending order, then printed as an array.

```
Run    FaultyMachine_MergeSorting

"C:\Users\Gian Carlo Trilles\PycharmProjects\pythonProject\.venv\Scripts\python.exe" "C:
Original machines: [303, 101, 404, 202, 101, 303, 404]
Sorted machines using merge sort: [101, 101, 202, 303, 303, 404, 404]
Faulty machine: 202

Process finished with exit code 0
```